Group Members:
John Mark Pardo
Byron Boral
Aerold Steven Reyes

DATA TRANSFORMATION AND ANALYSIS

Code:

```
import os
import sys

# ------------------------------------------------------------------------
# 0. CONFIGURE WINDOWS PYSPARK ENVIRONMENT
# ------------------------------------------------------------------------
os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    avg,
    col,
    count,
    date_format,
    regexp_replace,
    round,
    trim,
    when,
)

# ------------------------------------------------------------------------
# 1. INITIALIZE SPARK SESSION
# ------------------------------------------------------------------------
spark = (
    SparkSession.builder.appName("CollegeStudentsFloodRiskAnalysis")
    .master("local[*]")
    .config("spark.driver.host", "127.0.0.1")
    .getOrCreate()
)

spark.sparkContext.setLogLevel("ERROR")
```

```
# ------------------------------------------------------------------------
# 2. LOAD AND INSPECT DATASET (.XLSX)
# ------------------------------------------------------------------------
raw_xlsx_path = (
   "Assessing College Students Awareness and Preparedness Toward Flood Risks"
   " (Responses).xlsx"
)

# Ingestion using pandas, then parallelizing into PySpark DataFrame
pdf_raw = pd.read_excel(raw_xlsx_path)

# Impute empty open-ended survey answers
pdf_raw = pdf_raw.fillna(
   {
       pdf_raw.columns[22]: "None",
       pdf_raw.columns[23]: "None",
   }
)

df_raw = spark.createDataFrame(pdf_raw)

print("=== RAW SCHEMA & ROW COUNT ===")
print(f"Total Responses: {df_raw.count()}")
df_raw.printSchema()

# ------------------------------------------------------------------------
# 3. DATA CLEANING & TRANSFORMATION
# ------------------------------------------------------------------------
# Rename survey columns to standard identifier names
df_renamed = (
   df_raw.withColumnRenamed("Age", "age")
   .withColumnRenamed("Sex", "sex")
   .withColumnRenamed(df_raw.columns[5], "year_level")
   .withColumnRenamed(df_raw.columns[6], "course")
   .withColumnRenamed(df_raw.columns[7], "living_arrangement")
   .withColumnRenamed(df_raw.columns[8], "municipality")
   .withColumnRenamed(df_raw.columns[9], "flood_history")
   # Likert items: Awareness (1 to 5)
   .withColumnRenamed(df_raw.columns[10], "aw_warning_signals")
   .withColumnRenamed(df_raw.columns[11], "aw_monitor_updates")
   .withColumnRenamed(df_raw.columns[12], "aw_residence_risk")
   .withColumnRenamed(df_raw.columns[13], "aw_emergency_contacts")
   .withColumnRenamed(df_raw.columns[14], "aw_evac_routes")
   .withColumnRenamed(df_raw.columns[15], "aw_health_risks")
   # Likert items: Preparedness (1 to 5)
   .withColumnRenamed(df_raw.columns[16], "prep_go_bag")
   .withColumnRenamed(df_raw.columns[17], "prep_food_water")
   .withColumnRenamed(df_raw.columns[18], "prep_secure_docs")
   .withColumnRenamed(df_raw.columns[19], "prep_evac_plan")
```

```
    .withColumnRenamed(df_raw.columns[20], "prep_follow_protocols")
    .withColumnRenamed(df_raw.columns[21], "prep_breaker_cutoff")
    .withColumnRenamed(df_raw.columns[22], "biggest_challenge")
    .withColumnRenamed(df_raw.columns[23], "support_needed")
    # Drop redundant/consent/PII columns as taught in lecture slides
    .drop(
        "Do you voluntarily agree to participate in this study?",
        "Name (Optional):",
        "Email Address",
    )
)

df_clean = (
    df_renamed.withColumn(
        "age", regexp_replace(col("age"), "[\u2013\u2014]", "-")
    )
    .withColumn(
        "Timestamp", date_format(col("Timestamp"), "MM/dd/yyyy HH:mm:ss")
    )
    .withColumn("year_level", trim(col("year_level")))
    .withColumn("living_arrangement", trim(col("living_arrangement")))
    # Correct inconsistent formatting and typos in academic course names
    .withColumn(
        "course",
        when(
            col("course").rlike("(?i)system|bsis"),
            "BS Information Systems",
        )
        .when(
            col("course").rlike("(?i)criminology"),
            "BS Criminology",
        )
        .otherwise("BS Information Technology"),
    )
    # Standardize and clean messy municipality names
    .withColumn(
        "municipality",
        when(col("municipality").rlike("(?i)daet|camambugan"), "Daet")
        .when(
            col("municipality").rlike("(?i)panganiban|jpang"), "Jose Panganiban"
        )
        .when(col("municipality").rlike("(?i)talisay"), "Talisay")
        .when(col("municipality").rlike("(?i)mercedes"), "Mercedes")
        .when(col("municipality").rlike("(?i)labo"), "Labo")
        .when(col("municipality").rlike("(?i)basud"), "Basud")
        .when(col("municipality").rlike("(?i)vinzon"), "Vinzons")
        .when(col("municipality").rlike("(?i)paracale"), "Paracale")
        .when(col("municipality").rlike("(?i)vicente"), "San Vicente")
        .when(col("municipality").rlike("(?i)lorenzo"), "San Lorenzo Ruiz")
```

```
        .when(col("municipality").rlike("(?i)capalonga"), "Capalonga")
        .otherwise("Unspecified"),
    )
    # Feature Engineering: Compute Composite Likert Scale Indices (Averages)
    .withColumn(
        "awareness_score",
        round(
            (
                col("aw_warning_signals")
                + col("aw_monitor_updates")
                + col("aw_residence_risk")
                + col("aw_emergency_contacts")
                + col("aw_evac_routes")
                + col("aw_health_risks")
            )
            / 6.0,
            2,
        ),
    )
    .withColumn(
        "preparedness_score",
        round(
            (
                col("prep_go_bag")
                + col("prep_food_water")
                + col("prep_secure_docs")
                + col("prep_evac_plan")
                + col("prep_follow_protocols")
                + col("prep_breaker_cutoff")
            )
            / 6.0,
            2,
        ),
    )
)

# ------------------------------------------------------------------------
# 4. FIVE PYSPARK DATA ANALYSES
# ------------------------------------------------------------------------
print("\n" + "=" * 50)
print("ANALYSIS 1: Overall Mean Awareness vs. Preparedness")
print("=" * 50)
df_clean.select(
    round(avg("awareness_score"), 2).alias("overall_avg_awareness"),
    round(avg("preparedness_score"), 2).alias("overall_avg_preparedness"),
).show()

print("\n" + "=" * 50)
print("ANALYSIS 2: Preparedness by Living Arrangement")
```

```
print("=" * 50)
df_clean.groupBy("living_arrangement").agg(
    count("living_arrangement").alias("respondent_count"),
    round(avg("preparedness_score"), 2).alias("avg_preparedness"),
    round(avg("prep_go_bag"), 2).alias("avg_go_bag_readiness"),
).orderBy(col("avg_preparedness").desc()).show(truncate=False)

print("\n" + "=" * 50)
print("ANALYSIS 3: Awareness and Preparedness by Flood Experience History")
print("=" * 50)
df_clean.groupBy("flood_history").agg(
    count("flood_history").alias("count"),
    round(avg("awareness_score"), 2).alias("avg_awareness"),
    round(avg("preparedness_score"), 2).alias("avg_preparedness"),
).orderBy(col("avg_preparedness").desc()).show(truncate=False)

print("\n" + "=" * 50)
print("ANALYSIS 4: Awareness & Preparedness across College Year Levels")
print("=" * 50)
df_clean.groupBy("year_level").agg(
    count("year_level").alias("total_students"),
    round(avg("awareness_score"), 2).alias("avg_awareness"),
    round(avg("preparedness_score"), 2).alias("avg_preparedness"),
).orderBy("year_level").show()

print("\n" + "=" * 50)
print(
    "ANALYSIS 5: High Risk Students (Flood-Prone Areas Lacking Emergency"
    " Supplies)"
)
print("=" * 50)
high_risk_df = df_clean.filter(
    (
        col("flood_history").rlike("(?i)frequently")
        | col("flood_history").rlike("(?i)occasionally")
    )
    & ((col("prep_go_bag") <= 2) | (col("prep_food_water") <= 2))
).select(
    "sex",
    "year_level",
    "living_arrangement",
    "flood_history",
    "prep_go_bag",
    "prep_food_water",
    "preparedness_score",
)
print(f"Number of vulnerable students found: {high_risk_df.count()}")
high_risk_df.show(truncate=False)
```

```
# ============================================================================
# 5. REFINED, PRESENTATION-READY VISUALIZATIONS (DYNAMIC COUNTS)
# ============================================================================
pandas_df = df_clean.toPandas()

# ----------------------------------------------------------------------------
# CHART 1: Awareness vs. Preparedness by Living Arrangement
# ----------------------------------------------------------------------------
clean_living = {
    "Living with family / parents": "With Family",
    "Staying in a boarding house / dormitory / rented room": (
        "Dorm / Boarding House"
    ),
    "Living with relatives": "With Relatives",
}
pandas_df["living_group"] = (
    pandas_df["living_arrangement"].str.strip().map(clean_living)
)

living_summary = (
    pandas_df.groupby("living_group")
    .agg(
        avg_awareness=("awareness_score", "mean"),
        avg_preparedness=("preparedness_score", "mean"),
        n_count=("awareness_score", "count"),
    )
    .reindex(["Dorm / Boarding House", "With Family", "With Relatives"])
    .reset_index()
)

living_summary["display_label"] = (
    living_summary["living_group"]
    + "\n(n="
    + living_summary["n_count"].astype(str)
    + ")"
)

fig, ax = plt.subplots(figsize=(8.5, 5), dpi=300)
bar_width = 0.32
x = np.arange(len(living_summary))

rects1 = ax.bar(
    x - bar_width / 2,
    living_summary["avg_awareness"],
    bar_width,
    label="Awareness Score",
    color="#1d70b8",
)
rects2 = ax.bar(
```

```
    x + bar_width / 2,
    living_summary["avg_preparedness"],
    bar_width,
    label="Preparedness Score",
    color="#e66101",
)

for rect in rects1:
    h = rect.get_height()
    ax.annotate(
        f"{h:.2f}",
        xy=(rect.get_x() + rect.get_width() / 2, h),
        xytext=(0, 5),
        textcoords="offset points",
        ha="center",
        va="bottom",
        fontsize=10.5,
        fontweight="bold",
        color="#1d70b8",
    )

for rect in rects2:
    h = rect.get_height()
    ax.annotate(
        f"{h:.2f}",
        xy=(rect.get_x() + rect.get_width() / 2, h),
        xytext=(0, 5),
        textcoords="offset points",
        ha="center",
        va="bottom",
        fontsize=10.5,
        fontweight="bold",
        color="#c44e00",
    )

ax.set_title(
    "Flood Risk: Awareness vs. Preparedness by Living Arrangement",
    fontsize=13,
    fontweight="bold",
    pad=15,
)
ax.set_xticks(x)
ax.set_xticklabels(living_summary["display_label"], fontsize=11)
ax.set_ylabel("Mean Rating (Likert Scale 1-5)", fontsize=11)
ax.set_ylim(1.0, 5.0)
ax.axhline(
    3.0,
    color="gray",
    linestyle=":",
```

```
    linewidth=1.2,
    alpha=0.7,
    label="Neutral Midpoint (3.0)",
)

ax.legend(
    frameon=True,
    facecolor="white",
    framealpha=0.9,
    edgecolor="lightgray",
    loc="upper right",
)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.yaxis.grid(True, linestyle="--", alpha=0.35)
ax.set_axisbelow(True)

plt.tight_layout()
plt.savefig("viz1_awareness_vs_preparedness.png", dpi=300)
plt.show()

# ---------------------------------------------------------------------------
# CHART 2: Average Preparedness across Flood Experience Levels (Horizontal)
# ---------------------------------------------------------------------------
clean_flood = {
    "Never flooded": "Never Flooded",
    (
        "Rarely flooded (Floods only during extreme storm surges/historical"
        " events)"
    ): "Rarely Flooded",
    "Occasionally flooded (Floods only during severe typhoons)": (
        "Occasionally Flooded"
    ),
    "Frequently flooded (Floods even during regular heavy rain)": (
        "Frequently Flooded"
    ),
}
pandas_df["flood_group"] = (
    pandas_df["flood_history"].str.strip().map(clean_flood)
)

flood_order = [
    "Never Flooded",
    "Rarely Flooded",
    "Occasionally Flooded",
    "Frequently Flooded",
]

flood_summary = (
```

```
    pandas_df.groupby("flood_group")
    .agg(
        avg_preparedness=("preparedness_score", "mean"),
        n_count=("preparedness_score", "count"),
    )
    .reindex(flood_order)
    .reset_index()
)

flood_summary["display_label"] = (
    flood_summary["flood_group"]
    + " (n="
    + flood_summary["n_count"].astype(str)
    + ")"
)

fig, ax = plt.subplots(figsize=(9, 4.8), dpi=300)
y = np.arange(len(flood_summary))

teal_gradient = ["#80cdc1", "#35978f", "#01665e", "#003c30"]
bars = ax.barh(
    y,
    flood_summary["avg_preparedness"],
    height=0.55,
    color=teal_gradient,
    edgecolor="none",
)

for bar in bars:
    w = bar.get_width()
    ax.annotate(
        f"{w:.2f} / 5.0",
        xy=(w, bar.get_y() + bar.get_height() / 2),
        xytext=(8, 0),
        textcoords="offset points",
        ha="left",
        va="center",
        fontsize=10.5,
        fontweight="bold",
        color="#222222",
    )

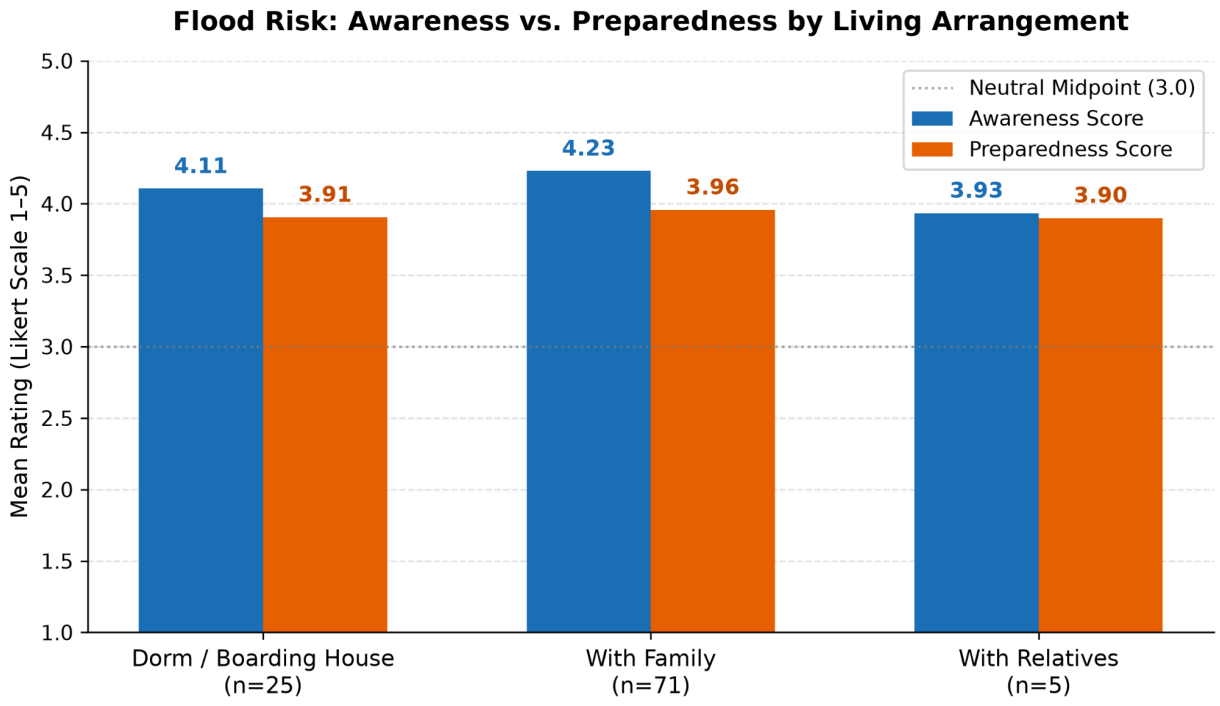
ax.set_yticks(y)
ax.set_yticklabels(flood_summary["display_label"], fontsize=11)
ax.set_xlabel("Average Preparedness Score (Likert 1-5)", fontsize=11)
ax.set_title(
    "Average Preparedness Level by Flood Exposure History",
    fontsize=13,
    fontweight="bold",
```

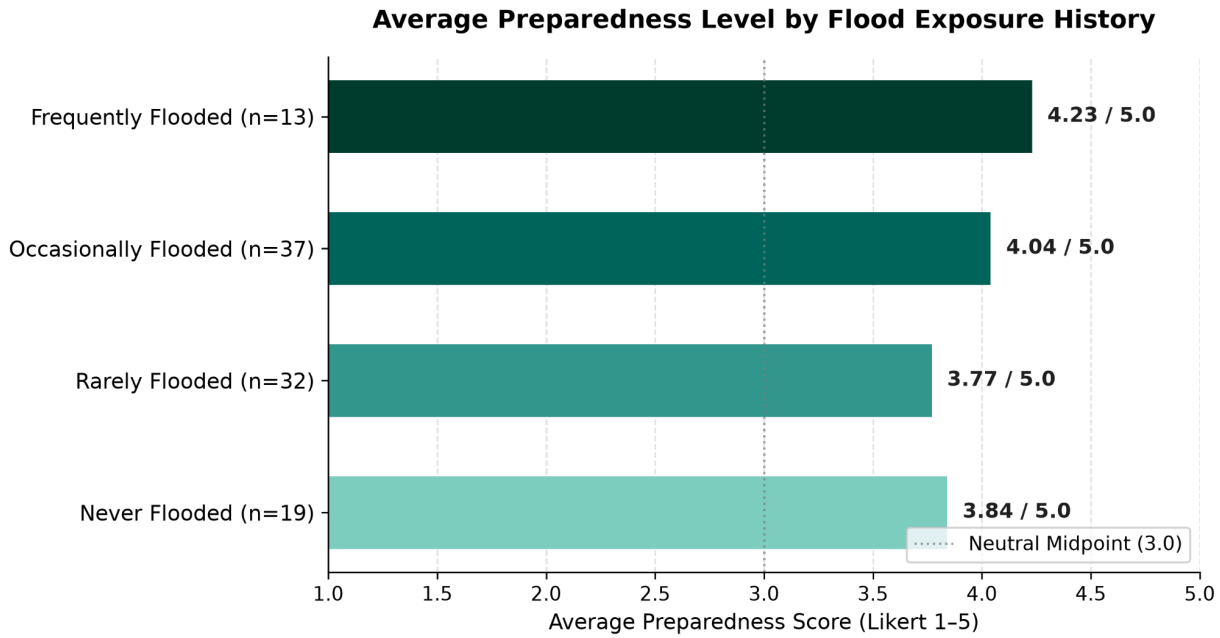
```
    pad=15,
)
ax.set_xlim(1.0, 5.0)
ax.axvline(
    3.0,
    color="gray",
    linestyle=":",
    linewidth=1.2,
    alpha=0.7,
    label="Neutral Midpoint (3.0)",
)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.xaxis.grid(True, linestyle="--", alpha=0.35)
ax.set_axisbelow(True)
ax.legend(frameon=True, loc="lower right")

plt.tight_layout()
plt.savefig("viz2_flood_experience.png", dpi=300)
plt.show()

# ------------------------------------------------------------------------
# 6. EXPORT PROCESSED DATASET
# ------------------------------------------------------------------------
pandas_df.to_csv(
    "processed_flood_risk_data.csv", index=False, encoding="utf-8-sig"
)
print(
    "\nExported clean processed dataset to 'processed_flood_risk_data.csv'."
)
```

Visualization from the code:





Findings:

College students show a grasp of weather warnings and flood risks with an overall awareness score of 4.18 out of 5.0. When it comes to taking real action the score drops to 3.94 out of 5.0. Looking at tasks students feel most confident with immediate steps like turning off the main electrical circuit breaker, which scored 4.31. They also feel prepared to protect items like academic records and valuable documents by keeping them in waterproof containers scoring 4.20. Following evacuation plans from their barangay or school is another area where they feel ready with a score of 4.18. However the biggest gap appears in emergency readiness. Keeping a stocked emergency Go-Bag gets a score of only 3.45. Similarly having a three-day supply of -perishable food and clean drinking water scores 3.53. These numbers suggest that while students know what to do in a disaster they often delay preparing the supplies needed. The gap between knowing and doing is clear. Awareness is high. Material preparation remains a work in progress.

With the expanded sample cohort of one hundred and one students, those living with parents or family students living with parents, n = 71 achieved the composite preparedness score of 3.96 and the highest awareness score of 4.23. Students living independently in boarding houses, dormitories or rented rooms students living independently in boarding houses n = 25 reported a preparedness score of 3.91 and a Go-Bag readiness of 3.40. Meanwhile students staying with relatives students staying with relatives, n = 5 averaged 3.90 in preparedness yet recorded the highest individual Go-Bag readiness at 3.80. These results illustrate that while family homes provide structural and resource security during severe weather students living outside parental care prioritize mobile survival items.

Personal exposure to flood events directly influences how people take action before a disaster. Students who live in areas that get flooded often (Flooded, n=13) had the highest scores in the whole group when it came to knowing about floods (4.55) and being ready for them (4.23). People living in places where flooding happens during big typhoons (Occasionally Flooded, n=37) came next with a preparedness score of 4.04. On the other hand students in places where flooding doesn't happen much (Rarely Flooded, n=32) had the lowest readiness score (3.77) which was even lower than students in areas that never flood (3.84, n=19). This shows that people in areas where flooding's rare tend to feel safe and don't prepare as much. This attitude is called "disaster complacency." Students in low-frequency flood zones often don't think the risk is real and don't take action as much as those who see flooding often.

Disaster readiness usually improves as students grow older in school. Disaster readiness was lowest among First-Year students (n=6) who had an awareness score of 3.86 and a preparedness score of 3.75. Disaster readiness rises noticeably in Second-Year students, who scored 3.97 on preparedness. In the Third Year Disaster readiness levels stay steadier with a preparedness score of 3.78. Disaster readiness peaks, for Fourth-Year seniors (n=61) who reached a 4.27 awareness score and a 4.00 preparedness score. Upperclassmen gain from knowing more about municipal disaster channels spending more time on campus and seeing clearer coordination plans when typhoons cause class suspensions.

A targeted multi-conditional query was run to identify students living in hazard zones specifically in areas that are frequently and occasionally flooded. This search uncovered seven students who are facing serious gaps in their emergency preparedness. Each of these students scored 2 on either their Go-Bag readiness or their 3-day supply of emergency food and drinking water. This means they are not fully equipped to handle a disaster. Among these seven one is a Second-Year student living in a flood zone. This student's overall preparedness score is especially low dropping to 2.83, which's below the neutral baseline. That shows a risk and a need for urgent support. These seven students are a high-priority group. They should be the focus of university-led efforts to improve student welfare. They also need access to community disaster kits. Should be included in local government unit emergency relief plans. Their situation calls for action.

Group Members Contributions:

1. John Mark Pardo. Managed the end-to-end data pipeline. Started by integrating Pandas to ingest the raw Excel survey responses. Converted them into an Apache Spark DataFrame. Inspected the schema structure and verified row counts. Systematically performed data cleaning. Dropped PII columns such, as consent, participant names and email addresses. Resolved formatting inconsistencies. Used expressions to standardize misspelled course titles. Normalized age dash characters. Cleaned municipality names across Camarines Norte. Oversaw the execution of all five PySpark DataFrame operations. Formulated the interpretations. Wrote the findings.

2. Byron Boral. Handled PySpark Transformations. Handled Query Analysis. Formulated composite Likert feature engineering. Created awareness_score. Created preparedness_score. Did groupBy. Did agg. Did orderBy. Did filter.

3. Aerold Reyes. Developed the Matplotlib/Seaborn pipeline. Created Charts 1 and 2. Engineered dynamic category count labels. Handled timestamp conversion.